

# 移动应用开发实验报告

## 封面信息

|      |                           |
|------|---------------------------|
| 项目   | 内容                        |
| 实验名称 | 微信小程序开发                   |
| 实验编号 | d20301035104、d20301009204 |
| 学号   | 2023210594                |
| 姓名   | 赫宇豪                       |
| 班级   | 软件 232                    |
| 日期   | 2026.4.11                 |
| 组别   | 组三 cifms                  |
| 项目名称 | 云端智能畜牧养殖管理系统              |
| 技术栈  | 微信小程序 (JavaScript)        |

## 实验目的

掌握微信小程序开发流程

学会 WXML/WXSS/JS 开发

理解页面路由与数据传递

掌握本地存储使用

体验 AI 辅助开发

## 实验环境

|    |    |    |
|----|----|----|
| 工具 | 版本 | 用途 |
|----|----|----|

| 工具      | 版本   | 用途             |
|---------|------|----------------|
| 微信开发者工具 | 最新版  | 小程序开发与调试       |
| Node.js | 18+  | 可选，用于 npm 依赖管理 |
| AI 辅助工具 | TRAE | 代码生成与调试        |

实验步骤

1. 需求分析

1.1 页面流程图

首页(通知列表) → 详情页(通知详情)  
↓  
本地存储(已读状态)

1.2 数据模型

```
// 通知数据结构
{
  id: 1,
  title: "动物健康检查通知",
  content: "详细内容...",
  author: "系统",
  time: "2024-01-01",
  isRead: false
}
```

2. 创建小程序项目

2.1 新建项目

打开微信开发者工具  
新建小程序项目  
填写 AppID（使用测试号）  
选择 JavaScript 模板

3. 通知列表页面开发

3.1 WXML 布局

```
<!-- pages/index/index.wxml -->
<view class="container">
```

```

<view class="news-list">
  <view
    class="news-item {{item.isRead ? 'read' : ''}}"
    wx:for="{{newsList}}"
    wx:key="id"
    bindtap="goToDetail"
    data-id="{{item.id}}"
  >
    <view class="title">{{item.title}}</view>
    <view class="info">
      <text>{{item.author}}</text>
      <text>{{item.time}}</text>
    </view>
  </view>
</view>

```

### 3.2 WXSS 样式

```

/* pages/index/index.wxss */
.news-item {
  padding: 16px;
  border-bottom: 1px solid #eee;
  background: #fff;
}
.news-item.read {
  opacity: 0.6;
}
.title {
  font-size: 16px;
  font-weight: bold;
  margin-bottom: 8px;
}
.info {
  font-size: 12px;
  color: #999;
  display: flex;
  justify-content: space-between;
}

```

### 3.3 JS 逻辑

```

// pages/index/index.js
Page({
  data: {
    newsList: []
  },

  onLoad() {
    this.loadNews();
  },

```

```

onPullDownRefresh() {
  this.loadNews().then() => {
    wx.stopPullDownRefresh();
  });
},

loadNews() {
  // 模拟数据
  const newsList = [
    { id: 1, title: "动物健康检查通知", content: "详细内容...", author: "系统", time: "2024-01-01", isRead: false },
    { id: 2, title: "智能监控系统升级通知", content: "详细内容...", author: "系统", time: "2024-01-02", isRead: false },
    { id: 3, title: "养殖记录更新提醒", content: "详细内容...", author: "系统", time: "2024-01-03", isRead: false }
  ];

  // 读取本地存储的已读状态
  const readIds = wx.getStorageSync('readIds') || [];
  newsList.forEach(news => {
    news.isRead = readIds.includes(news.id);
  });

  this.setData({ newsList });
},

goToDetail(e) {
  const id = e.currentTarget.dataset.id;
  wx.navigateTo({
    url: `/pages/detail/detail?id=${id}`
  });
}
});

```

### 3.4 配置文件

```

// pages/index/index.json
{
  "navigationBarTitleText": "云端智能畜牧养殖管理系统",
  "enablePullDownRefresh": true,
  "usingComponents": {}
}

```

## 4. 详情页开发

### 4.1 WXML 布局

```

<!-- pages/detail/detail.wxml -->
<view class="container">

```

```
<view class="title">{{news.title}}</view>
<view class="info">
  <text>发布部门: {{news.author}}</text>
  <text>发布时间: {{news.time}}</text>
</view>
<view class="content">{{news.content}}</view>
</view>
```

## 4.2 JS 逻辑

```
// pages/detail/detail.js
```

```
Page({
  data: {
    news: {}
  },

  onLoad(options) {
    const id = parseInt(options.id);
    this.loadDetail(id);
  },

  loadDetail(id) {
    // 模拟数据
    const allNews = [
      { id: 1, title: "动物健康检查通知", content: "本次健康检查安排如下...", author: "系统", time: "2024-01-01" },
      { id: 2, title: "智能监控系统升级通知", content: "系统升级通知...", author: "系统", time: "2024-01-02" },
      { id: 3, title: "养殖记录更新提醒", content: "养殖记录更新提醒...", author: "系统", time: "2024-01-03" }
    ];

    const news = allNews.find(n => n.id === id);
    this.setData({ news });

    // 标记为已读
    this.markAsRead(id);
  },

  markAsRead(id) {
    let readIds = wx.getStorageSync('readIds') || [];
    if (!readIds.includes(id)) {
      readIds.push(id);
      wx.setStorageSync('readIds', readIds);
    }
  }
});
```

## 5. 本地存储功能

5.1 已读状态存储

```
// 保存已读ID
wx.setStorageSync('readIds', [1, 2, 3]);

// 读取已读ID
const readIds = wx.getStorageSync('readIds');

// 检查是否已读
const isRead = readIds.includes(newsId);
```

6. 测试与发布

6.1 真机预览

编译项目，扫描二维码预览，测试各项功能

6.2 体验版发布

点击上传，填写版本信息，获取体验版二维码

实验结果

验证结果

| 功能     | 状态 | 备注 |
|--------|----|----|
| 通知列表展示 | 成功 | 无  |
| 详情页跳转  | 成功 | 无  |
| 已读状态存储 | 成功 | 无  |
| 下拉刷新   | 成功 | 无  |
| 真机预览   | 成功 | 无  |

截图记录

通知列表页面



通知详情页面



问题与解决

| 问题描述         | 解决方案                     | 解决结果 |
|--------------|--------------------------|------|
| 真机运行时导航栏显示异常 | 移除自定义导航栏，使用微信默认导航栏       | 问题解决 |
| 徽章显示不完整      | 添加 badge-text 类，确保文本正确显示 | 问题解决 |

| 问题描述       | 解决方案                          | 解决结果 |
|------------|-------------------------------|------|
| 快速操作按钮布局混乱 | 使用 flexbox 布局代替 grid 布局，提高兼容性 | 问题解决 |

实验总结

实验收获

- 1.掌握了微信小程序的基本开发流程
- 2.学习了使用 WXML、WXSS、JavaScript 进行小程序开发
- 3.理解了小程序页面路由与数据传递机制
- 4.掌握了本地存储的使用方法
- 5.体验了 AI 辅助开发工具在代码生成中的作用

技术要点

- 1.小程序页面布局与样式设计
- 2.数据绑定与列表渲染
- 3.页面路由与参数传递
- 4.本地存储的使用
- 5.下拉刷新功能实现

考核指标

| 考核项   | 评分标准              | 得分   |
|-------|-------------------|------|
| 功能完整性 | 通知列表、详情页、本地存储功能完整 | 30 分 |
| 代码质量  | 代码结构清晰，注释完整       | 20 分 |
| 页面设计  | 界面美观，用户体验良好       | 15 分 |
| 本地存储  | 已读状态存储正常          | 15 分 |
| 真机预览  | 小程序可在真机上正常运行      | 10 分 |

| 考核项  | 评分标准        | 得分   |
|------|-------------|------|
| 实验报告 | 报告结构完整，内容详实 | 10 分 |

**教师评价**

| 项目     | 评价 |
|--------|----|
| 实验完成情况 |    |
| 技术掌握程度 |    |
| 创新点    |    |
| 综合评价   |    |
| 成绩     |    |

教师签名：\_\_\_\_\_ 日期：\_\_\_\_\_